Ryan Ramdihal
EEL 4742
Lab 10: Advanced Timer Features

Introduction:

This lab utilizes multiple channels of the timer module, using the timing output patterns to drive a Pulse Width Modulation signal to a GPIO pin, to change the brightness levels of the LED. The first objective is to use 2 channels of a timer to schedule interrupts toggling the LED. Then, use a 3rd channel to make the 2 channels toggling the LED to skip every other 4 seconds. By driving a PWM signal to the red LED, we can change the brightness by changing the TACCR1 value.

10.1:Timer's Multiple Channels

The objective of this code is to set up two periodic interrupts and toggle the red and green LEDs when to their respective interrupt routines. Using a 32KHz ACLK divided by 4, to cause an interrupt every 0.1 seconds and 0.5 seconds for the red and green LEDs to toggle. Given the code template to complete the missing parts: Starting with configuring channel 0 by finding the # of cycles channel 0 needs for the interrupt to start every 0.1 seconds. Values are found by the equation: (cycles*(ID/freq) = time. Plugging in the timer,ID, and frequency, the TA0CCR0 value should be 819. Moving on to the configuration of channel 1 timer controls, finding the # of cycles is 4096 for a 0.5 second time period. Then starting the ACLK timer with a divider by 4 in continuous mode. Followed by enabling low power mode 3 (only ACLK on) to save power. Lastly, is to write the ISR for channel 1, where we toggle the green LED, schedule the next interrupt by adding 4096 to the trigger limit, and finally clearing the channel 1 interrupt flag since it's a shared vector. Additionally, I noticed that the A0 vector's scheduling value is incorrect; it should add 819 instead of 3277 due to the divider being 4, not 1.

10.1 Code:

```
#include <msp430fr6989.h>
#define redLED BIT0 // Red at P1.0
#define greenLED BIT7 // Green at P9.7
void config_ACLK_to_32KHz_crystal() {
  // By default, ACLK runs on LFMODCLK at 5MHz/128 = 39 KHz
  // Reroute pins to LFXIN/LFXOUT functionality
  PJSEL1 &= ~BIT4;
  PJSEL0 |= BIT4;
  // Wait until the oscillator fault flags remain cleared
  CSCTL0 = CSKEY; // Unlock CS registers
  do {
     CSCTL5 &= ~LFXTOFFG; // Local fault flag
     SFRIFG1 &= ~OFIFG; // Global fault flag
  }
  while((CSCTL5 & LFXTOFFG) != 0);
     CSCTL0_H = 0; // Lock CS registers
return;
}
void main(void) {
   WDTCTL = WDTPW | WDTHOLD; // Stop WDT
   PM5CTL0 &= ~LOCKLPM5; // Enable GPIO pins
```

```
    P1DIR |= redLED;
    P9DIR |= greenLED;
    P1OUT &= ~redLED;
    P9OUT &= ~greenLED;
   // config_ACLK_to_32KHz_crystal();
    // Configure Channel 0
    TA0CCR0 = 818; // @ 32 KHz   x*(4/32khz) = 0.1 sec -> x = 819
    TA0CCTL0 |= CCIE;
    TA0CCTL0 &= ~CCIFG;
    // Configure Channel 1
    TA0CCR1 = 4095; // @32 KHz   y*(4/32khz) = 0.5 sec -> y= 4096
    TA0CCTL1 |= CCIE;
    TA0CCTL1 &= ~CCIFG;
    // Configure timer (ACLK) (divide by 4) (continuous mode)
    TA0CTL = TASSEL_1 | ID_2 | MC_2| TACLR;
    // Engage a low-power mode
    _low_power_mode_3(); //only ACLK is on
    return;
}
// ISR of Channel 0 (A0 vector)
#pragma vector = TIMER0_A0_VECTOR
__interrupt void T0A0_ISR() {
    P1OUT ^= redLED; // Toggle the red LED
    TA0CCR0 += 818; // Schedule the next interrupt //ask TA given code had 3277,but dividing by 4,not 1?
    // Hardware clears Channel 0 flag
}
// ISR of Channel 1 (A1 vector)
#pragma vector = TIMER0_A1_VECTOR
__interrupt void T0A1_ISR() {
    P9OUT ^= greenLED; // Toggle the green LED
    TA0CCR1 += 4095; // Schedule the next interrupt
    // Clear Channel 1 interrupt flag
    TA0CCTL1 &= ~CCIFG;
}
```

10.2:Using Three Channels

In the updated code, both LEDs should turn off every alternate 4-second interval while still toggling every 0.1 and 0.5 seconds when they're on. Channel 2 is utilized to set up the 4-second interrupt, which controls the toggling routines for channels 0 and 1. A status variable is declared to track whether the 4-second interval should have the LEDs off (status 0), on (status 1), or back off. An if statement checks the Channel 2 flag, representing the 4-second intervals, and the status value to determine whether the LEDs should be on or off.

10.2 Code

```
#include <msp430fr6989.h>
#define redLED BIT0 // Red at P1.0
#define greenLED BIT7 // Green at P9.7
int status = 0;
void config_ACLK_to_32KHz_crystal() {
  // By default, ACLK runs on LFMODCLK at 5MHz/128 = 39 KHz
  // Reroute pins to LFXIN/LFXOUT functionality
```

```
  PJSEL1 &= ~BIT4;
  PJSEL0 |= BIT4;
  // Wait until the oscillator fault flags remain cleared
  CSCTL0 = CSKEY; // Unlock CS registers
  do {
     CSCTL5 &= ~LFXTOFFG; // Local fault flag
     SFRIFG1 &= ~OFIFG; // Global fault flag
  }
  while((CSCTL5 & LFXTOFFG) != 0);
     CSCTL0_H = 0; // Lock CS registers
return;
}
void main(void) {
   WDTCTL = WDTPW | WDTHOLD; // Stop WDT
   PM5CTL0 &= ~LOCKLPM5; // Enable GPIO pins
   P1DIR |= redLED;
   P9DIR |= greenLED;
   P1OUT &= ~redLED;
   P9OUT &= ~greenLED;
   config_ACLK_to_32KHz_crystal();
   // Configure Channel 0
   TA0CCR0 = 818; // @ 32 KHz x*(4/32khz) = 0.1 sec
   TA0CCTL0 |= CCIE;
   TA0CCTL0 &= ~CCIFG;
   // Configure Channel 1
   TA0CCR1 = 4095; // @32 KHz y*(4/32khz) = 0.5 sec
   TA0CCTL1 |= CCIE;
   TA0CCTL1 &= ~CCIFG;
   // Configure Channel 2
      TA0CCR2 = 32767; // @32 KHz z*(4/32khz) = 4 sec
      TA0CCTL2 |= CCIE;
      TA0CCTL2 &= ~CCIFG;
   // Configure timer (ACLK) (divide by 4) (continuous mode)
   TA0CTL = TASSEL_1 | ID_2 | MC_2;
   // Engage a low-power mode
   _low_power_mode_3(); //only ACLK is on
   return;
}
// ISR of Channel 0 (A0 vector)
#pragma vector = TIMER0_A0_VECTOR
__interrupt void T0A0_ISR() {
   P1OUT ^= redLED; // Toggle the red LED
   TA0CCR0 += 818; // Schedule the next interrupt
   // Hardware clears Channel 0 flag
}
// ISR of Channel 1 (A1 vector)
#pragma vector = TIMER0_A1_VECTOR
__interrupt void T0A1_ISR() {
   if((TA0CCTL1 & CCIFG)!= 0){
      P9OUT ^= greenLED; // Toggle the green LED
      TA0CCR1 += 4095; // Schedule the next interrupt
   // Clear Channel 1 interrupt flag
   TA0CCTL1 &= ~CCIFG;
   }
```

```
  if ((TA0CCTL2 & CCIFG != 0) && (status == 0)){ //LEDs off
     TA0CCTL0 &= ~CCIE;
     TA0CCTL1 &= ~CCIE;
     P1OUT &= ~redLED;
     P9OUT &= ~greenLED;
     TA0CCR2 += 32767;   // schedule next interrupt
        status = 1;
  }
  if((TA0CCTL2 & CCIFG != 0) && (status == 1)){ // LEDs turn on
     TA0CCR0 = 818; // 0.1 seconds //resets
     TA0CCTL0 |= CCIE; // turn on enable bit
     TA0CCR1 = 4095; // 0.5 seconds // resets
     TA0CCTL1 |= CCIE; // turn on enable bit
     //clear channel 0 interrupt
     TA0CCTL0 &= ~CCIFG;
    //clear channel 1 interrupt
     TA0CCTL1 &= ~CCIFG;
     // clear channel 2 interrupt
     TA0CCTL2 &= ~CCIFG;
     status = 0;
        }
}
```

10.3:Driving a PWM Signal on the Pin

Using Timer A, output patterns can be driven to interact with pins without CPU intervention. By configuring the pins to Timer A0 the redLED pin, and the channel, we are able to change the brightness of the redLED. By changing the TA0CCR1 value from 0-32, the brightness level will be adjusted. By including the ADC function in lab 8, I was able to implement the joystick's direction to change the brightness of the LED.

10.3 Code:

```
// Setting up a PWM on P1.0 (red LED)
// P1.0 coincides with TA0.1 (Timer0_A Channel 1)
// Configure P1.0 pin to TA0.1 ---> P1DIR=1, P1SEL1=0, P1SEL0=1
// PWM frequency: 1000 Hz -> 0.001 seconds
#include <msp430fr6989.h>
#define PWM_PIN BIT0
#define FLAGS UCA1IFG // Contains the transmit & receive flags
#define RXFLAG UCRXIFG // Receive flag
#define TXFLAG UCTXIFG // Transmit flag
#define TXBUFFER UCA1TXBUF // Transmit buffer
#define RXBUFFER UCA1RXBUF // Receive buffer
#define BUT1 BIT1 // P1.1
#define BUT2 BIT2 // P1.2
#define redLED BIT0 // Red at P1.0
#define greenLED BIT7 // Green at P9.7
unsigned int ascii[10] = {48, 49, 50, 51, 52, 53, 54, 55, 56, 57};
```

```
void Initialize_ADC();
void Initialize_UART();
void config_ACLK_to_32KHz_crystal();
void uart_write_char(unsigned char ch);
void uart_write_string(char string[]);
void uart_write_uint16(unsigned int num);
unsigned int get_numSize(unsigned int num);
void parse_int_into_array(unsigned int num, unsigned int numSize, unsigned int* digits);
int main(void)
{
  WDTCTL = WDTPW | WDTHOLD;   // stop watchdog timer
  PM5CTL0 &= ~LOCKLPM5; // Enable GPIO pins
  P1DIR |= PWM_PIN; // P1DIR bit = 1
  P1SEL1 |= ~PWM_PIN; // P1SEL1 bit = 0
  P1SEL0 |= PWM_PIN; // P1SEL0 bit = 1
  config_ACLK_to_32KHz_crystal();
  Initialize_ADC();
  Initialize_UART();
  TA0CCTL1 =OUTMOD_7; // Modify OUTMOD field to 7
  // (ACLK @ 32 KHz) (Divide by 1) (Up mode)
  TA0CCR0 = 33; // @ 32 KHz --> 0.001 seconds (1000 Hz)
  TA0CTL = TASSEL_1 | ID_0 | MC_1 | TACLR;
    // Enable the global interrupt bit (call an intrinsic function)
    _enable_interrupts();
    TA0CCTL1 =OUTMOD_7; // Modify OUTMOD field to 7
    TA0CCR1 = 16; // Modify this value between 0 and 32 to adjust the brightness level
    // Starting the timer in the up mode; period = 0.001 seconds
  unsigned int xdata = 0;
  unsigned int ydata = 0;
  while (1){
    _delay_cycles(500000);
    ADC12CTL0 |= ADC12SC; //start conversion
    while ((ADC12CTL1 & ADC12BUSY) == ADC12BUSY){} //in conversion
    xdata = ADC12MEM0; //x axis
    ydata = ADC12MEM1; //y axis
    if(xdata <= 1024){
      TA0CCR1 -= 2; //decrement
    }
    if(xdata >= 3072){
      TA0CCR1 += 2; //increment
    }
    else if(ydata >= 3072){
      TA0CCR1 = 32;
    }
    else if(ydata <= 1024){
      TA0CCR1 = 0;
    }
    uart_write_string("X-Axis: ");
        uart_write_uint16(xdata);
        uart_write_string("  Y-Axis: ");
        uart_write_uint16(ydata);
        uart_write_char('\n');
        uart_write_char('\r');
        xdata = 0;
```

```
            ydata = 0;
    }
}
void Initialize_ADC() {
    // Divert the pins to analog functionality
    // X-axis: A10/P9.2, for A10 (P9DIR=x, P9SEL1=1, P9SEL0=1)
    P9SEL1 |= BIT2;
    P9SEL0 |= BIT2;
    // Y-axis: A4/P8.7, for A4 (P8DIR=x, P8SEL1=1, P8SEL0=1)
    P8SEL1 |= BIT7;
    P8SEL0 |= BIT7;
    // Turn on the ADC module
    ADC12CTL0 |= ADC12ON;
    // Turn off ENC (Enable Conversion) bit while modifying the configuration
    ADC12CTL0 &= ~ADC12ENC;
    //*************** ADC12CTL0 ***************
    // Set ADC12SHT0 (select the number of cycles that you determined)
    // 32;
    ADC12CTL0 |= ADC12SHT0_3;
    // Set the bit ADC12MSC (Multiple Sample and Conversion)
    ADC12CTL0 |= ADC12MSC;
    //*************** ADC12CTL1 ***************
    // Set ADC12SHS (select ADC12SC bit as the trigger)
    // 0;
    ADC12CTL1 |= ADC12SHS_0;
    // Set ADC12SHP bit
    //ADC12SHP = 1;
    ADC12CTL1 |= ADC12SHP;
    // Set ADC12DIV (select the divider you determined)
    ADC12CTL1 |= ADC12DIV_0;
    // Set ADC12SSEL (select MODOSC)
    ADC12CTL1 |= ADC12SSEL_0;
    // Set ADC12CONSEQ (select sequence-of-channels)
    ADC12CTL1 |= ADC12CONSEQ_1;
    //*************** ADC12CTL2 ***************
    // Set ADC12RES (select 12-bit resolution)
    ADC12CTL2 |= ADC12RES_2;
    // Set ADC12DF (select unsigned binary format)
    ADC12CTL2 &= ~ADC12DF;
    //*************** ADC12CTL3 ***************
    // Set ADC12CSTARTADD to 0 (first conversion in ADC12MEM0)
    ADC12CTL3 |= ADC12CSTARTADD_0;
    //*************** ADC12MCTL0 ***************
    // Set ADC12VRSEL (select VR+=AVCC, VR-=AVSS)
    ADC12MCTL0 |= ADC12VRSEL_0;
    // Set ADC12INCH (select channel A10)
    ADC12MCTL0 |= ADC12INCH_10;
    //*************** ADC12MCTL1 ***************
    // Set ADC12VRSEL (select VR+=AVCC, VR-=AVSS)
    ADC12MCTL1 |= ADC12VRSEL_0;
    // Set ADC12INCH (select the analog channel that you found)
    ADC12MCTL1 |= ADC12INCH_4;
    // Set ADC12EOS (last conversion in ADC12MEM1)
    ADC12MCTL1 |= ADC12EOS;
```

```
   // Turn on ENC (Enable Conversion) bit at the end of the configuration
   ADC12CTL0 |= ADC12ENC;
   return;
   }
void config_ACLK_to_32KHz_crystal() {
// By default, ACLK runs on LFMODCLK at 5MHz/128 = 39 KHz
// Reroute pins to LFXIN/LFXOUT functionality
PJSEL1 &= ~BIT4;
PJSEL0 |= BIT4;
// Wait until the oscillator fault flags remain cleared
CSCTL0 = CSKEY; // Unlock CS registers
do {
CSCTL5 &= ~LFXTOFFG; // Local fault flag
SFRIFG1 &= ~OFIFG; // Global fault flag
} while((CSCTL5 & LFXTOFFG) != 0);
CSCTL0_H = 0; // Lock CS registers
return;
}
void Initialize_UART(){
   // Configure pins to UART functionality
   P3SEL1 &=  ~(BIT4|BIT5);
   P3SEL0 |= (BIT4|BIT5);
   // Main configuration register
   UCA1CTLW0 = UCSWRST; // Engage reset; change all the fields to zero
   // Most fields in this register, when set to zero, correspond to the popular configuration
   UCA1CTLW0 |= UCSSEL_2; // Set clock to SMCLK
   // Configure the clock dividers and modulators (and enable oversampling)
   UCA1BRW = 6; // divider
   // Modulators: UCBRF = 8 = 1000 --> UCBRF3 (bit #3)
   // UCBRS = 0x20 = 0010 0000 = UCBRS5 (bit #5)
   UCA1MCTLW = UCBRF3 | UCBRS5 | UCOS16;
   // Exit the reset state
   UCA1CTLW0 &=  ~UCSWRST;
   return;
}
void uart_write_char(unsigned char ch){
// Wait for any ongoing transmission to complete
   while ( (FLAGS & TXFLAG)==0 ) {}
// Copy the byte to the transmit buffer
   TXBUFFER = ch; // Tx flag goes to 0 and Tx begins!
   return;
}
void uart_write_string(char string[]){
   int i = 0;
   while(string[i] != '\0'){
      uart_write_char(string[i]);
      i++;
   }
   uart_write_char(string[i]);
}
void uart_write_uint16(unsigned int n){
   int i;
      unsigned int numSize = 1; // Initialize numSize to 1
      unsigned int num = n; // Create a copy of n to avoid modifying the original value
```

```
    // Count number of digits
    while (num /= 10) {
        numSize++;
    }
    unsigned int digits[5]; // Number will not have more than 5 digits
    // Parse the numbers digit, place into declared array
    for (i = numSize - 1; i >= 0; i--) {
        digits[i] = n % 10;
        n = n / 10;
    }
    // Write each digit over UART
    for (i = 0; i < numSize; i++) {
        uart_write_char(ascii[digits[i]]);
    }
   // uart_write_char(10); // Write /n
    //uart_write_char(13); // Write /r
}
```

10.4:Timer Input Capture

10.4 Code:

Conclusion:

In conclusion, this lab effectively demonstrates the versatile capabilities of timer modules in microcontroller applications. By leveraging multiple channels of the timer module, we were able to schedule interrupts and toggle an LED with precise timing. Furthermore, integrating a third channel to modulate the timing of the LED toggling allowed us to implement more complex control patterns, such as skipping toggles every four seconds. Additionally, by utilizing pulse width modulation (PWM) signals, we could dynamically adjust the brightness of the LED by manipulating the TACCR1 value, and with ADC controls, implement the joystick to adjust the LED brightness.

Student Q&A:

1. Copy the description of P9.7 from the pinout diagram and determine whether this pin supports timer-based output.

Green LED
P9.7/ESICI3/A15/C15.
It does not support timer-based output for PWM.

2. In the code with three channels, why was it necessary to divide ACLK?

By doing so, it slows down the frequency for smaller time durations.

3. In the first part, we configured two periodic interrupts using two channels of the timer. Is this approach scalable? For example, using a Timer A module with five channels, can we configure five periodic interrupts? Explain and mention in what mode the timer would run.

Yes, we can configure 5 periodic interrupts. The timer would run in continuous mode.

4. As an example, Channel 1's interrupt ours every 40K cycles. The first interrupt is scheduled for when TAR=40K cycles. Explain how the next interrupt is scheduled? Explain the overflow mechanism and show why it results in a correct value.
The interrupt is scheduled by adding another 40K cycles to TACCR. The channel takes the difference from that new value, 80K, and the 16-bit limit to get the next time to run an interrupt.